

B1g_Mac

 **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

Here's a zip file.

Tools Used

- `7z` – Extract compressed files
 - `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read metadata
 - **Ghidra** – Reverse engineering and decompilation
 - **Python** – Custom extraction script
 - **x64dbg / x32dbg** – Windows debugger (alternative method)
-

Complete Solution

Step 1: Download and Extract

Download the file named `b1g_mac.zip` from the challenge page.

```
7z x b1g_mac.zip
```

After extraction, you will find:

- `main.exe` – A 32-bit Windows executable
- A folder named `test` containing **18 .bmp image files**

Step 2: Basic File Inspection

First, verify the file type:

```
file main.exe
```

Expected output:

```
main.exe: PE32 executable for MS Windows 4.00 (console), Intel i386, 15 sections
```

The binary is a **32-bit Windows console executable**.

Step 3: Search for Visible Text

```
strings main.exe | grep -i "picoCTF"
```

Output: (no results)

The flag isn't directly visible.

Step 4: Inspect Metadata

```
exiftool main.exe
```

Output:

```
ExifTool Version Number      : 13.50
File Name                    : main.exe
...
File Type                    : Win32 EXE
Machine Type                 : Intel 386 or later, and compatibles
Time Stamp                   : 2019:04:06 19:24:21+02:00
Entry Point                   : 0x14e0
Subsystem                     : Windows command line
```

Nothing suspicious in the metadata.

Step 5: Initial Execution Attempt

Running the executable locally fails because it expects a `flag.txt` file (only present on the remote server).

Create a dummy flag file to satisfy the file read requirement:

```
echo picoCTF{fake_flag} > flag.txt
```

Now run the executable again:

```
./main.exe
```

Output:

```
Work is done!  
Wait for 5 seconds to exit.
```

The program prints a message and waits 5 seconds before exiting. This suggests the program performs some operation (likely hiding data in the BMP files) before displaying this message.

Step 6: Reverse Engineering with Ghidra

Install Ghidra if not already installed:

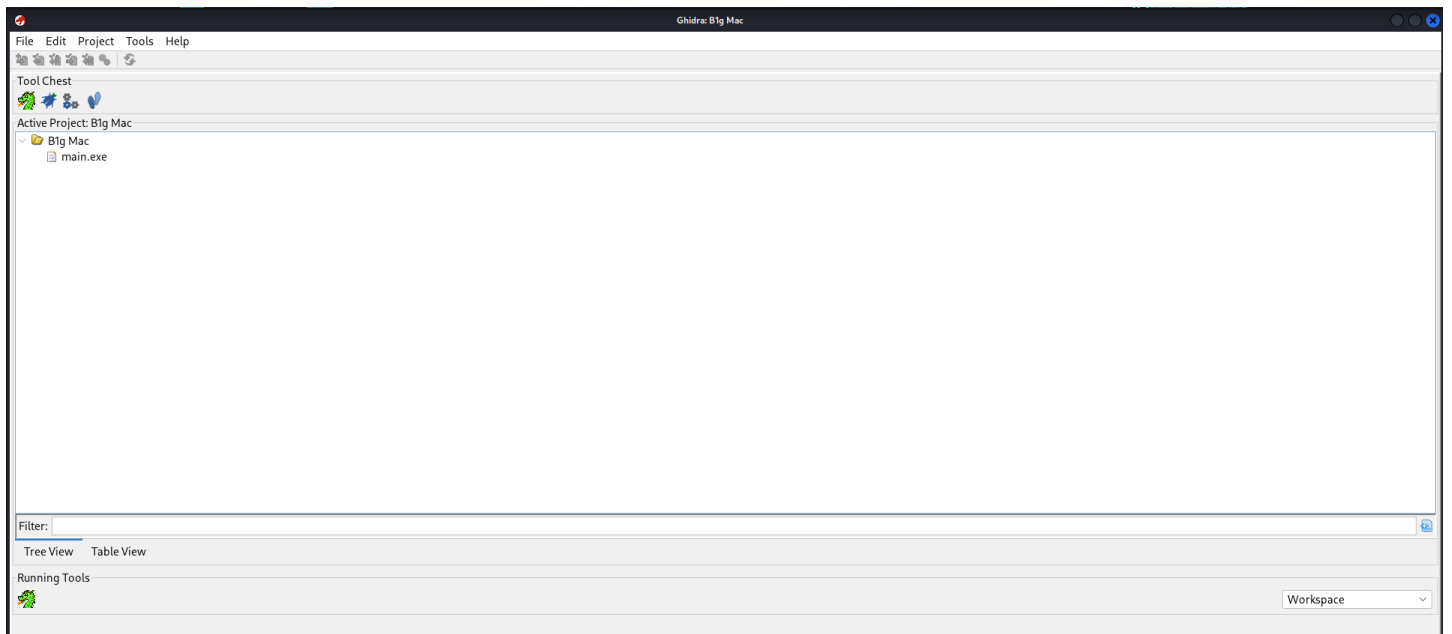
```
apt install ghidra
```

Launch Ghidra:

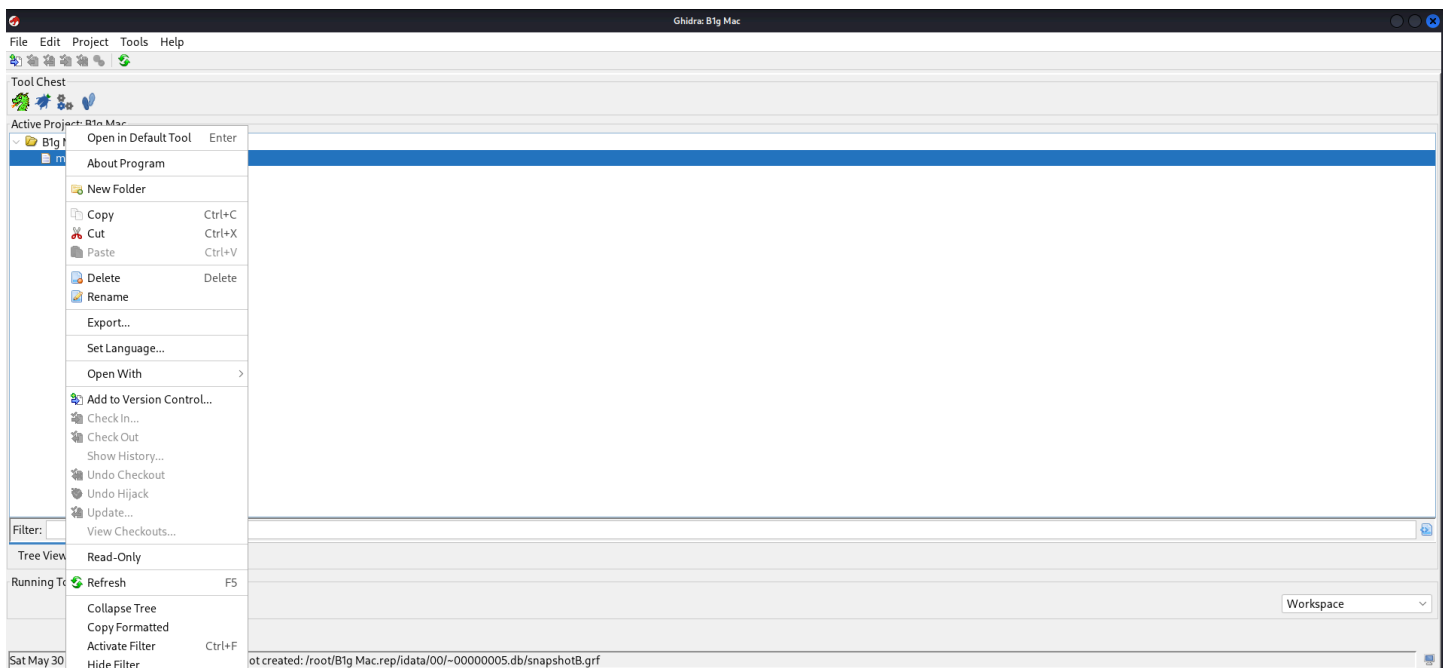
```
ghidra
```

Steps:

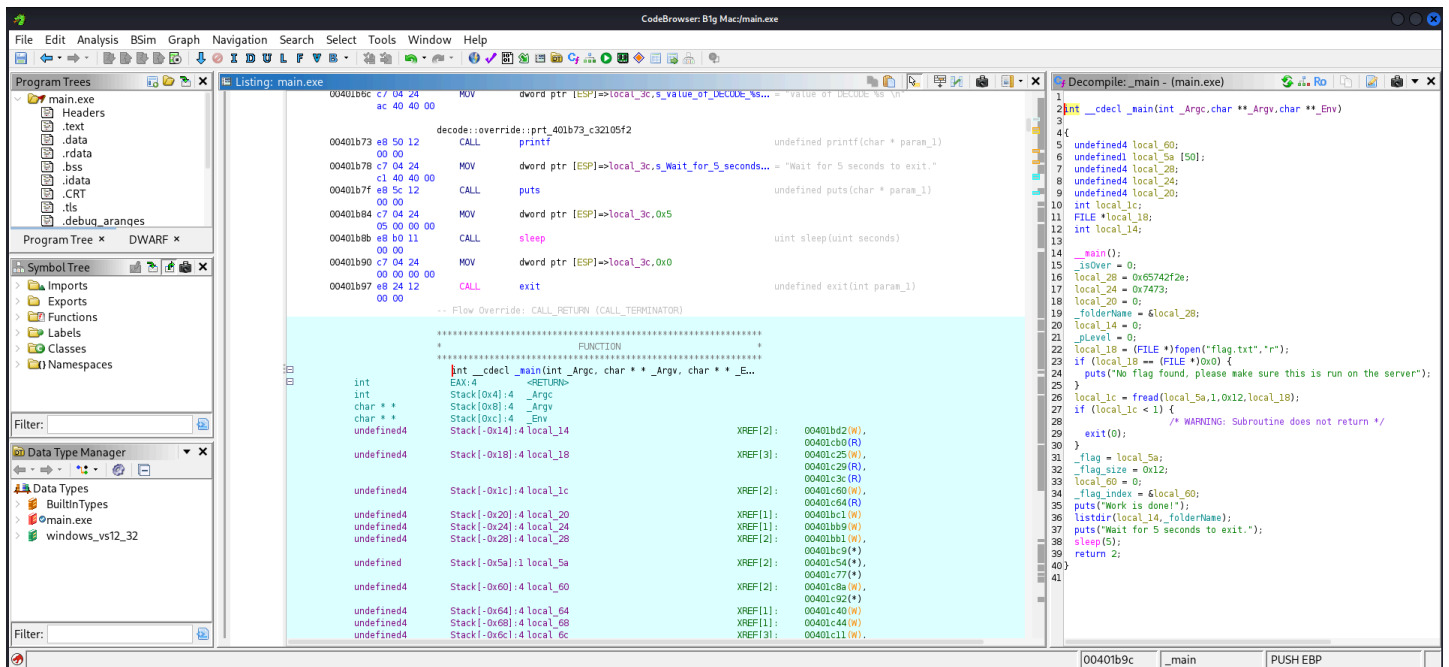
1. Create a new project
2. Import the file `main.exe`



3. Right-click on `main.exe` and select **Open in Default Tool**



4. Click **Yes** to analyze the binary and select the default analysis tools



5. The decompiled `main` function will appear on the left panel

Step 7: Analyze the Main Function

The program:

1. Opens `flag.txt` and reads **18 characters**
2. Initializes some global variables
3. Calls `listdir` (after printing "Work is done!" – strangely, not much happens before that message)

Step 8: Examine `listdir` Function

Right-click `listdir` → **References** → **Show Call Trees For listdir**

The function reveals three call paths; one of them is an uncalled function.

Key observations from `listdir` :

- `_listdir` is a recursive function that iterates over files in the `./test` folder
- When `param_1 == 0`, it calls `hideInFile` for alternating files (the "Copy" files)
- When `param_1 == 1`, it calls `decodeBytes` on the files instead

Step 9: Examine `hideInFile` Function

```
void __cdecl hideInFile(LPCSTR param_1)
{
    // Opens the file
    // Reads 2 bytes from the flag (stored in _flag global variable)
    // Calls encodeBytes() to hide those bytes in the file timestamps
    // Uses different timestamps depending on _pLevel (0, 1, or 2)
}
```

C

How it works:

- Takes 2 characters from the flag
 - Encodes them into the `dwLowDateTime` fields of the file's timestamps
 - The flag is split across multiple files in the `test` folder
-

Step 10: Examine `decodeBytes` Function

```
void __cdecl decodeBytes(LPCSTR param_1)
{
    // Opens the file
    // Reads the timestamps (LastWriteTime, LastAccessTime, CreationTime)
    // Extracts the encoded bytes from the dwLowDateTime fields
    // Appends them to a global buffer (_buff)
    // Continues until the flag is fully reconstructed
}
```

C

When `decode` is called (which in turn calls `listdir(1, _folderName)`), it:

- Reads the encoded timestamps from the BMP files
 - Extracts the hidden characters
 - Reconstructs the original flag
-

Step 11: Python Extraction Script

Instead of debugging, we can extract the flag directly from the ZIP archive. The flag is stored in the **NTFS Extra Field** of the ZIP's Central Directory, specifically in the `LastWriteTime` field of each

BMP file.

python

```
#!/usr/bin/python3 -u

import os

filename = "b1g_mac.zip"
fileindex = 0
statePhase = 0

centralDirSignature = b'PK\x01\x02'
bmpCopyFileName = b'Copy'
bmpCopyFileExte = b'.bmp'
flag = ''

with open(filename, 'rb') as fh:
    while (fourbytes := fh.read(4)):
        if (statePhase == 0) and (fourbytes == centralDirSignature):
            print(f'Found a Central Directory Signature at {hex(fileindex)}')
            statePhase = 1
            fileindex = fileindex + 4
        elif (statePhase == 1) and (fourbytes == bmpCopyFileName):
            print(f'    * "Copy" at {hex(fileindex)}')
            statePhase = 2
            fileindex = fileindex + 4
        elif (statePhase == 2) and (fourbytes == bmpCopyFileExte):
            print(f'    * ".bmp" at {hex(fileindex)}')
            statePhase = 3
            # Skip to the NTFS Extra Field where LastWriteTime is stored
            fileindex = fileindex + 16
        elif (statePhase == 3):
            # Extract two bytes (note: Little-endian order)
            flag += chr(fourbytes[1])
            flag += chr(fourbytes[0])
            print(f'    Got : "{chr(fourbytes[1])}" and "{chr(fourbytes[0])}" around {hex(fileindex)}')
            statePhase = 0
            fileindex = fileindex + 2
        else:
            fileindex = fileindex + 1

    fh.seek(fileindex, os.SEEK_SET)

print('\nFlag is : {}'.format(flag))
```

Output:

```
Found a Central Directory Signature at 0x0
* "Copy" at 0x26
* ".bmp" at 0x2a
Got : "p" and "i" around 0x58
Found a Central Directory Signature at 0x5c
* "Copy" at 0x82
* ".bmp" at 0x86
Got : "c" and "o" around 0xb4
...
Flag is : picoCTF{<REDACTED>}
```

Step 12: Alternative Method – Debugging on Windows

If you prefer debugging over a Python script:

Using x32dbg (Windows only)

1. **Important:** Extract the ZIP file using **7-zip** (Windows built-in extraction changes file timestamps and breaks the decoding)
2. Run `x96dbg` from the x64dbg suite and select **x32dbg**
3. Open `main.exe` in x32dbg
4. Set a breakpoint at `0x00401bda` :

```
SetBPX 00401bda
```

5. Click **Run** until the breakpoint is hit
6. Change the EIP to `0x00401afe` (to execute the decode function):

```
EIP = 00401afe
```

7. Click **Run** one more time
8. The flag will be displayed in the console output

Console output:

```
value of DECODE picoCTF{<REDACTED>}
Wait for 5 seconds to exit.
```

 Final Result

picoCTF{<REDACTED>}

Note: The actual flag is redacted here. In the real challenge, the Python script or debugging method will reveal the complete flag.

 Key Concepts

Concept	Description
NTFS Extra Field	A section in ZIP Central Directory entries that stores additional metadata (timestamps, permissions).
LastWriteTime	File timestamp that can be used to hide small amounts of data (2 bytes per file).
ZIP Central Directory	Section at the end of a ZIP file containing metadata for each entry.
Ghidra	Reverse engineering framework for decompiling binaries to C-like pseudocode.
Steganography via Timestamps	Hiding data in file timestamps (LastWriteTime, LastAccessTime, CreationTime).
x32dbg	Windows debugger for 32-bit executables.

 Pro Tips

- Always check the ZIP Central Directory** – Hidden data is often stored there, not in the file contents themselves.
- Use 7-zip for extraction** – Windows built-in extraction modifies file timestamps, which can break steganography that relies on them.

3. **Look for `Copy` in filenames** – The script extracts data from files named `CopyX.bmp` where `X` is a number.
4. **The flag is stored in little-endian order** – Notice the script reads `fourbytes[1]` first, then `fourbytes[0]`.
5. **Ghidra is powerful for reversing** – It can decompile binaries even without symbols or debugging.
6. **The `_pLevel` variable controls which timestamps are used** – Different files may hide data in different timestamp fields.

Alternative Tools

Tool	Purpose
Ghidra	Decompilation and static analysis
x64dbg / x32dbg	Windows dynamic debugging
Python	Automated extraction from ZIP
Wine + GDB	Linux-based debugging (less reliable)

Conclusion

The challenge `big_mac.zip` contained an executable (`main.exe`) and a folder of BMP images. The executable reads a `flag.txt` file and hides its characters in the file timestamps (NTFS Extra Field) of the BMP files inside the ZIP archive.

Instead of debugging the executable (which requires Windows or a debugger), we wrote a Python script to read the ZIP's Central Directory, locate the `CopyX.bmp` entries, and extract the two-byte sequences from the `LastWriteTime` field. These bytes, when combined in order, formed the flag.

This challenge demonstrates:

- **ZIP file forensics** – Examining the Central Directory for hidden data
- **Timestamps as data carriers** – Using file metadata to hide information

- **Reverse engineering** – Understanding the hiding mechanism via Ghidra
- **Automated extraction** – Scripting the recovery process without running the binary

Key takeaway: Data can be hidden not only in file contents but also in **metadata** like timestamps. The ZIP Central Directory's NTFS Extra Field is a common location for such steganography. Always examine the ZIP structure with a hex viewer or custom script when faced with suspicious archives.